GUIDE COMPLET SUR LA CONTENEURISATION

Gérer les conteneurs avec Docker

Lancement, ports, volumes, copie de fichiers, variables, Dockerfile, réseaux, Docker Compose et déploiements multi-conteneurs

Version de révision : schémas explicatifs · termes expliqués · points clés · corrigé des questions

Objectif

Apprendre à gérer des conteneurs avec Docker et être capable de réaliser des tâches pratiques : lancement de conteneurs, port mapping, volumes, copie de fichiers, variables d'environnement, construction d'images avec un Dockerfile, réseaux, puis déploiement d'applications multi-conteneurs avec Docker Compose.

> **En clair — Idée clé**
> Tout le chapitre répond à quatre questions : **quoi lancer ?** (image, Dockerfile) · **comment l'atteindre ?** (ports) · **où garder les données ?** (volumes, cp) · **avec qui parle-t-il ?** (réseaux, variables, Compose).

Comment lire ce document

Bloc	Signification
En clair	Explication simple d'un terme important, avec une analogie.
À retenir	Les points clés de la section, à connaître par cœur.
Attention	Pièges, erreurs fréquentes et nuances.
Conseil examen	Réflexe à avoir pour ne pas se tromper en contrôle.
Blocs de code	Commandes à tester dans un terminal avec Docker installé.

Sommaire

0. Introduction à Docker . . . 3
1. Construire une image avec un Dockerfile . . . 4
2. Lancer un conteneur . . . 6
3. Port Mapping . . . 7
4. Volumes avec -v . . . 8
5. Les principaux types de montages . . . 9
6. Copier des fichiers avec docker cp . . . 11
7. Variables d'environnement . . . 12
8. ARG vs ENV . . . 13
9. Exemple ARG → ENV . . . 14
10. Les réseaux Docker . . . 15
11. Docker Compose : le multi-conteneurs simplifié . . . 17
12. Autres commandes essentielles à connaître . . . 18
Ce qu'il faut absolument retenir . . . 20
Glossaire des termes importants . . . 21
Questions de compréhension — avec corrigés . . . 22

0. Introduction à Docker

Docker est la plateforme de conteneurs la plus répandue. Son interface en ligne de commande est celle que Podman a reprise : les commandes sont donc presque identiques.

COMMANDES PRINCIPALES

```
docker run
docker ps
docker exec
docker cp
docker build
docker network
docker volume
docker compose
```

Dans ce chapitre, on se concentre sur les commandes utiles pour administrer des applications conteneurisées, puis on termine avec **Docker Compose**, l'outil qui décrit toute une application multi-conteneurs dans un seul fichier.

> **En clair — Architecture client / daemon**
> Quand vous tapez `docker run`, vous parlez à un **client** (la commande `docker`) qui envoie votre demande, par une API, à un **service permanent** : le daemon `dockerd`. C'est lui qui fait le vrai travail (images, conteneurs, réseaux, volumes) en s'appuyant sur `containerd` et `runc`. Analogie : le client est le serveur en salle qui prend la commande ; le daemon est la cuisine qui prépare le plat.

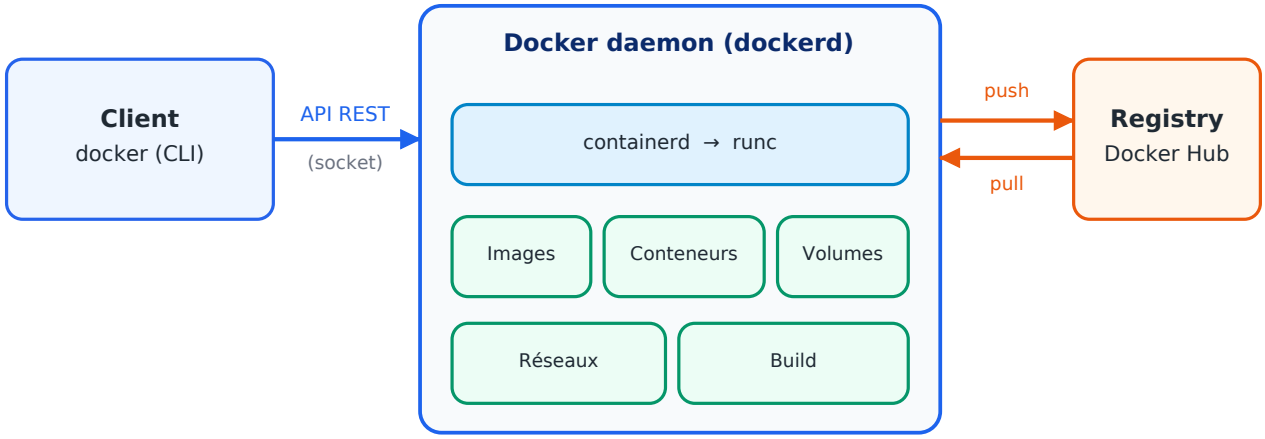


Figure 1 — Le client docker envoie ses ordres au daemon, qui télécharge (pull) ou publie (push) les images sur un registry.

Docker et Podman : ce qui change

Critère	Docker	Podman
Architecture	Daemon permanent (`dockerd`)	Sans daemon (daemonless)
Utilisateur	Daemon exécuté en root par défaut (mode rootless disponible)	Rootless natif
Fichier de build	`Dockerfile`	`Containerfile` (le `Dockerfile` est accepté)
Multi-conteneurs	Docker Compose (`docker compose`)	`podman compose`, pods, Quadlet
Registry par défaut	Docker Hub (`docker.io`)	Demande de choisir selon la configuration

En clair — Noms d'images courts

Avec Docker, `nginx` signifie `docker.io/library/nginx:latest` : le registry (Docker Hub), l'organisation (`library`) et le tag (`latest`) sont complétés automatiquement.

Attention — Le groupe « docker » donne les droits root

Accéder au daemon Docker revient à avoir un accès root à la machine : un utilisateur du groupe `docker` peut, par exemple, monter tout le disque de l'hôte dans un conteneur. À n'accorder qu'à des personnes de confiance.

1. Construire une image avec un Dockerfile

Un Dockerfile décrit comment construire une image. Instructions importantes :

Instruction	Rôle	Exemple
`FROM`	Image de base	`FROM nginx:alpine`
`ARG`	Variable disponible pendant le build	`ARG APP_VERSION`
`ENV`	Variable d'environnement	`ENV APP_VERSION=1.0`
`RUN`	Exécuter une commande pendant le build	`RUN apk add --no-cache curl`
`COPY`	Copier des fichiers dans l'image	`COPY index.html /usr/share/nginx/html/`
`WORKDIR`	Définir le répertoire de travail	`WORKDIR /app`
`EXPOSE`	Documenter un port utilisé par l'application	`EXPOSE 80`
`CMD`	Commande par défaut	`CMD ["nginx", "-g", "daemon off;"]`
`ENTRYPOINT`	Programme principal du conteneur	`ENTRYPOINT ["curl"]`

Attention — EXPOSE ne publie rien

`EXPOSE 80` ne publie pas le port sur l'hôte. Pour le publier, il faut utiliser `-p`, par exemple `-p 8080:80`.

Exemple

DOCKERFILE

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

L'image de base `nginx:alpine` définit déjà la commande de démarrage de nginx : inutile de la répéter. Construire, puis lancer :

TERMINAL

```
docker build -t custom-nginx .
docker run -d --name custom-nginx -p 8080:80 custom-nginx
```

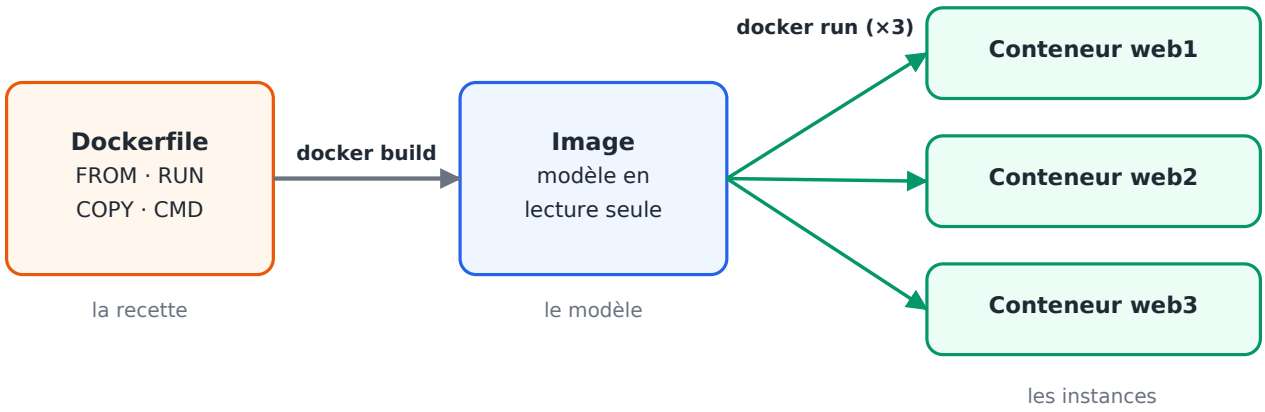


Figure 2 — Du Dockerfile au conteneur : build puis run. Une même image peut lancer plusieurs conteneurs.

En clair — Image vs conteneur

Une **image** est un moule (ou une recette) ; un **conteneur** est le gâteau fabriqué avec ce moule. Avec un seul moule, on peut faire autant de gâteaux qu'on veut. L'image est en lecture seule ; chaque conteneur ajoute sa propre petite couche modifiable.

En clair — Pourquoi « daemon off; » ?

Un conteneur s'arrête dès que son processus principal se termine. Par défaut, nginx se met en arrière-plan et son processus de départ se termine aussitôt : le conteneur s'arrêterait. `daemon off;` force nginx à rester au premier plan. L'image officielle contient déjà `CMD ["nginx", "-g", "daemon off;"]`.

CMD ou ENTRYPOINT ?

`CMD` définit la commande par défaut, facilement remplaçable à l'exécution. `ENTRYPOINT` définit le programme principal du conteneur, plus difficile à remplacer (il faut `--entrypoint`). On les combine souvent : ENTRYPOINT = le programme, CMD = ses arguments par défaut.

DOCKERFILE

```
FROM alpine:3.20
RUN apk add --no-cache curl
WORKDIR /app
ENTRYPOINT ["curl"]
CMD ["--version"]
```

TERMINAL

```
docker build -t mon-curl .
docker run --rm mon-curl                       # exécute : curl --version
docker run --rm mon-curl -I https://example.com   # CMD remplacé : curl -I ...
```

Cache de couches et .dockerignore

Chaque instruction crée une **couche** de l'image. Docker réutilise (cache) les couches inchangées : placez donc en premier ce qui change rarement (installation des dépendances) et en dernier ce qui change souvent (votre code). Le fichier `.dockerignore` liste ce qu'il ne faut pas envoyer au build (`.git`, `node_modules`, fichiers secrets…).

Attention

Le tag `latest` n'est pas une garantie de « dernière version » : il change dans le temps. Pour une image reproductible, fixez un tag précis (`alpine:3.20`).

À retenir

✓ Un Dockerfile = la recette de construction d'une image (`docker build`).
✓ `EXPOSE` documente un port, il ne le publie pas.
✓ Une image = un modèle ; un conteneur = une instance de cette image.

2. Lancer un conteneur

La commande principale est :

TERMINAL
```
docker run [OPTIONS] IMAGE [COMMANDE]
```

TERMINAL
```
docker run -d --name web nginx
```

Si l'image n'existe pas localement, Docker la télécharge, puis crée et démarre le conteneur.

Options importantes

Option	Rôle	Exemple
`-d`	Exécuter en arrière-plan (detached)	`docker run -d nginx`
`--name`	Donner un nom au conteneur	`--name web`
`-p`	Publier un port	`-p 8080:80`
`-v`	Monter un volume ou un répertoire	`-v $HOME/site:/data`
`-e`	Définir une variable d'environnement	`-e RESPONSE="Hello"`
`--network`	Connecter le conteneur à un réseau	`--network net`
`--rm`	Supprimer le conteneur à son arrêt	`--rm`
`-it`	Mode interactif avec un terminal	`docker run -it alpine sh`
`--restart`	Politique de redémarrage automatique	`--restart unless-stopped`

Vérifier les conteneurs :

TERMINAL
```
docker ps          # conteneurs en cours d'exécution
docker ps -a       # tous, y compris arrêtés
```

En clair — Mode détaché (-d)

Sans `-d`, le conteneur occupe votre terminal (premier plan), comme un programme ordinaire. Avec `-d`, il tourne en arrière-plan et le terminal vous est rendu, comme un service qu'on démarre puis qu'on laisse travailler.

À retenir

✓ `-d` arrière-plan · `--name` nom · `-p` port · `-v` volume · `-e` variable · `--network` réseau · `--rm` suppression auto.

✓ `docker ps` = conteneurs en cours ; `docker ps -a` = tous, y compris arrêtés.

3. Port Mapping

3.1. Principe

Un processus qui écoute dans un conteneur n'est pas automatiquement accessible sur un port de l'hôte. On utilise :

SYNTAXE

```
-p HOST_PORT:CONTAINER_PORT
```

TERMINAL

```
docker run -d \
  --name nginx \
  -p 8001:80 \
  nginx
```

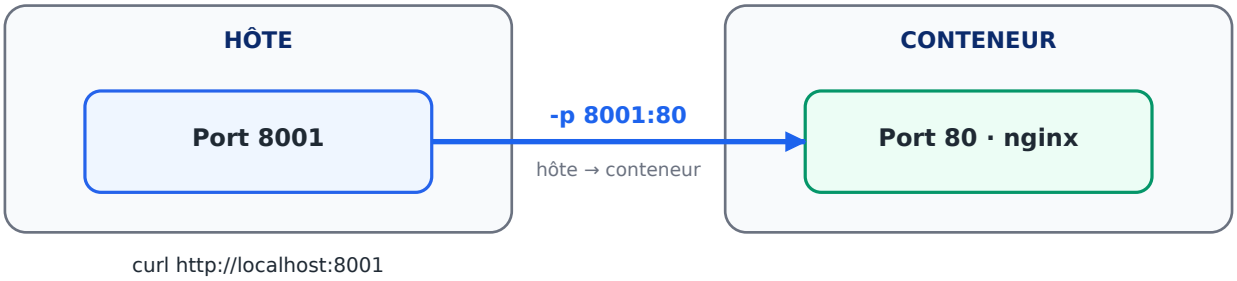


Figure 3 — Le port 8001 de l'hôte est redirigé vers le port 80 du conteneur.

On accède alors à Nginx avec :

TERMINAL

```
curl http://localhost:8001
```

En clair — Port mapping

Le conteneur est un bureau avec son propre standard téléphonique (son réseau isolé). Le port mapping, c'est créer un renvoi d'appel : « tout appel arrivant au poste 8001 de l'immeuble (l'hôte) est transféré au poste 80 du bureau (le conteneur) ».

Conseil examen

Toujours lire `HOST:CONTAINER` et non l'inverse : à gauche le port de l'hôte, à droite le port du conteneur.

Attention — Un port hôte, un seul conteneur

Deux conteneurs ne peuvent pas publier le même port de l'hôte : `-p 8001:80` deux fois provoque une erreur (port déjà utilisé). En revanche, ils peuvent tous les deux écouter sur le port 80 à l'intérieur, car chacun a son propre réseau. Il suffit de les publier sur des ports hôtes différents : `-p 8001:80` et `-p 8002:80`.

3.2. Variantes utiles

TERMINAL

```
docker run -d -p 127.0.0.1:8001:80 nginx   # joignable seulement depuis l'hôte
docker run -d -P nginx                     # publie les ports EXPOSE (ports aléatoires)
docker port nom-du-conteneur               # affiche les correspondances de ports
```

Attention — Docker et le pare-feu

Docker écrit lui-même ses règles réseau (iptables). Un port publié avec `-p 8001:80` peut rester joignable depuis l'extérieur même si un pare-feu comme `ufw` semble le bloquer. Pour limiter l'accès, publiez sur `127.0.0.1`.

À retenir

✓ `-p HOTE:CONTENEUR` : le premier nombre est sur la machine, le second dans le conteneur.
✓ Sans `-p`, l'application n'est pas joignable depuis l'hôte.

4. Volumes avec -v

Les conteneurs sont éphémères par nature. Si les données importantes restent uniquement dans le filesystem du conteneur, elles peuvent être perdues lorsque le conteneur est supprimé. Les volumes permettent de conserver ou de partager des données indépendamment du cycle de vie du conteneur.

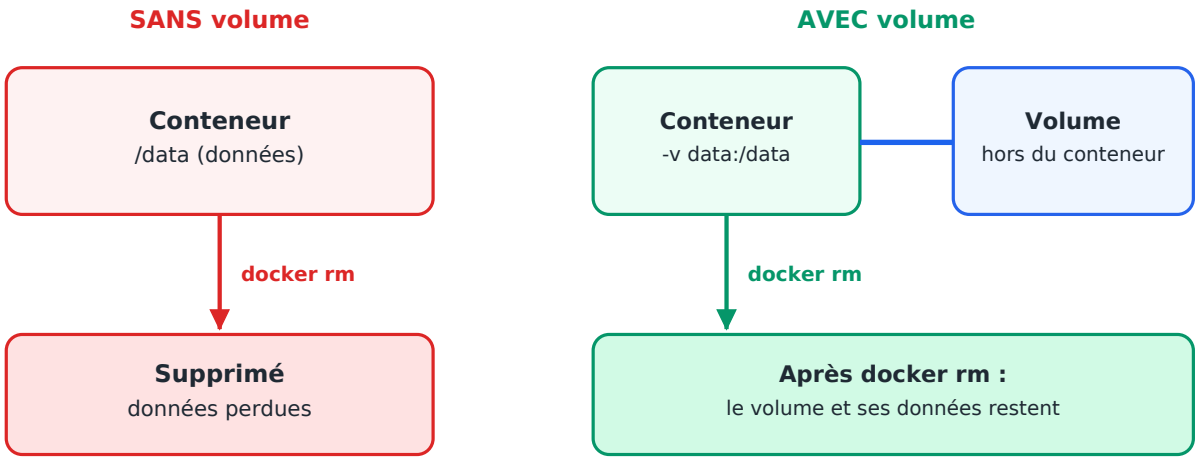


Figure 4 — Sans volume, les données disparaissent avec le conteneur ; avec un volume, elles restent.

La syntaxe générale est :

SYNTAXE

```
-v SOURCE:DESTINATION[:OPTIONS]
```

TERMINAL

```
-v /home/student/web:/usr/share/nginx/html
```

5. Les principaux types de montages

Avec Docker, on rencontre quatre cas pratiques : bind mount, named volume, anonymous volume et tmpfs.

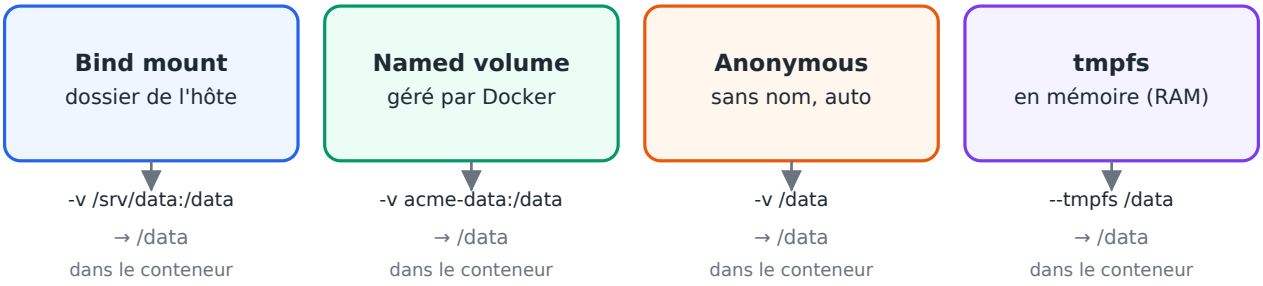


Figure 5 — Les types de montage et leur syntaxe.

5.1. Bind mount

On monte un répertoire ou un fichier existant de l'hôte. Syntaxe : `-v HOST_PATH:CONTAINER_PATH`.

TERMINAL

```
docker run -d \
  --name nginx \
  -v "$HOME/site:/usr/share/nginx/html:ro" \
  -p 8080:80 \
  nginx
```

Le contenu du répertoire de l'hôte est directement accessible dans le conteneur. L'option `:ro` (read-only) empêche le conteneur de le modifier.

> **Attention — Chemin inexistant**
>
> Avec `-v`, si le dossier source n'existe pas, Docker le **crée** (en root). Avec la syntaxe longue `--mount`, Docker renvoie une erreur : plus sûr pour détecter une faute de frappe.

SYNTAXE --MOUNT

```
docker run -d --name nginx \
  --mount type=bind,source="$HOME/site",target=/usr/share/nginx/html,readonly \
  nginx
```

Pourquoi :Z sur un système avec SELinux ?

Sur Fedora, RHEL ou Rocky Linux (SELinux actif), les fichiers de l'hôte portent un contexte de sécurité que le processus du conteneur n'est pas autorisé à lire, même si `chmod` / `chown` semblent corrects. L'option `:Z` demande à Docker de **relabelliser** le dossier avec un contexte adapté au conteneur.

TERMINAL

```
-v "$HOME/site:/usr/share/nginx/html:Z"
```

> **En clair — SELinux et relabel**
>
> SELinux est un second système de contrôle d'accès : même si la porte est déverrouillée par les permissions classiques (`chmod`), le vigile SELinux exige un badge portant la bonne étiquette. Relabelliser, c'est coller sur le dossier l'étiquette que le vigile accepte pour ce conteneur.

Option	Contexte SELinux	Usage
`:Z (majuscule)`	Privé	Un seul conteneur utilise le volume
`:z (minuscule)`	Partagé	Plusieurs conteneurs partagent le même contenu

> **Attention**
> Sur un système sans SELinux (Ubuntu, Debian), `:Z` et `:z` ne sont pas nécessaires. Ne relabellisez jamais un dossier système (`/home`, `/etc`...) : réservez `:Z` à des dossiers dédiés.

Quand utiliser un bind mount ? Développement, configuration, partage de fichiers, contenu web, scripts, données que l'administrateur souhaite gérer directement depuis l'hôte.

5.2. Named volume

Docker peut créer et gérer un volume. Créer, vérifier, puis monter le volume :

TERMINAL
```
docker volume create host-data
docker volume ls
docker run -d --name app -v host-data:/data nginx
```

Le volume est identifié par son **nom**, le stockage physique est géré par Docker. Si le volume n'existe pas, `docker run -v nom:/data` le crée automatiquement. Pour voir où il est stocké : `docker volume inspect host-data` (champ `Mountpoint`).

Quand l'utiliser ? Bases de données, données persistantes, applications qui doivent conserver leurs données, partage de données entre conteneurs.

5.3. Anonymous volume

TERMINAL
```
docker run -d --name app -v /data nginx
```

Docker crée alors un volume associé au conteneur, avec un identifiant aléatoire.

> **Attention — Volume sans nom**
> Un volume anonyme est difficile à reconnaître dans `docker volume ls` et facile à oublier. Pour des données importantes, préférez un named volume. `docker rm -v` supprime aussi les volumes anonymes du conteneur.

5.4. tmpfs

Un montage `tmpfs` vit **en mémoire vive** : rien n'est écrit sur le disque, et tout disparaît à l'arrêt du conteneur. Utile pour des données temporaires ou sensibles.

TERMINAL
```
docker run -d --name app --tmpfs /tmp nginx
```

5.5. Résumé

Type	Syntaxe	Source	Géré par
Bind mount	`-v /home/student/data:/data`	Chemin de l'hôte (existant)	L'administrateur
Named volume	`-v acme-data:/data`	Nom du volume	Docker
Anonymous volume	`-v /data`	Aucune (volume sans nom)	Docker
tmpfs	`--tmpfs /data`	Mémoire vive (RAM)	Docker

À retenir

- ✓ **Bind mount** : on expose un dossier précis de l'hôte.
- ✓ **Named volume** : Docker gère le stockage, on le retrouve par son nom.
- ✓ **Anonymous volume** : un volume sans nom, créé automatiquement.
- ✓ **tmpfs** : stockage temporaire en mémoire.
- ✓ Pour une base de données, utiliser un volume pour ne pas perdre les données.

6. Copier des fichiers avec docker cp

La commande `docker cp` permet de copier des fichiers ou répertoires **Host → Container** et **Container → Host**. Elle fonctionne aussi sur un conteneur arrêté.



Figure 6 — docker cp : la partie contenant « NOM: » désigne toujours le conteneur.

6.1. Host → Container

```
TERMINAL
docker cp index.html nginx:/usr/share/nginx/html/index.html
```

6.2. Container → Host

```
TERMINAL
docker cp nginx:/etc/nginx/nginx.conf ./nginx.conf
```

6.3. Copier un répertoire

```
TERMINAL
docker cp "$HOME/workspace/acme-nginx-web/html/." acme-demo-nginx:/usr/share/nginx/html/
```

Le `/.` est utile lorsque l'on souhaite copier le **contenu** du répertoire plutôt que créer un sous-répertoire `html` dans la destination.

6.4. cp ou -v ?

	docker cp	-v (volume / bind mount)
Nature	Copie ponctuelle (instantané)	Montage permanent
Données	Deux copies indépendantes	Mêmes données visibles des deux côtés
Moment	À tout moment, sur un conteneur existant	Défini à la création du conteneur
Persistance	Perdu si le conteneur est supprimé (sauf copie vers l'hôte)	Conservé après suppression du conteneur

> **En clair — cp vs -v**
> `docker cp`, c'est envoyer une photocopie d'un document : après l'envoi, chacun modifie sa copie dans son coin. `-v`, c'est partager le même classeur : toute modification est vue immédiatement par les deux.

7. Variables d'environnement

Une variable d'environnement permet de transmettre une configuration au processus exécuté dans le conteneur. Avec Docker : `-e NAME=value`.

TERMINAL
```
docker run -d --name app -e RESPONSE="Hello ACME" alpine sleep 3600
```

Pour vérifier depuis l'hôte :

TERMINAL
```
docker exec app sh -c 'echo "$RESPONSE"'
# Hello ACME
```

> **Attention — Guillemets simples ou doubles ?**
> Écrivez la commande entre **guillemets simples** : avec des guillemets doubles, c'est le shell de l'hôte qui remplacerait `$RESPONSE` (variable inconnue, donc vide) avant même d'envoyer la commande au conteneur.

Plusieurs variables : --env-file

Quand il y a beaucoup de variables, on les met dans un fichier `NOM=valeur` (une par ligne) :

TERMINAL
```
docker run -d --name app --env-file ./app.env alpine sleep 3600
docker inspect -f '{{.Config.Env}}' app    # voir les variables d'un conteneur
```

8. ARG vs ENV

C'est une distinction très importante lors de la construction d'une image.

8.1. ARG

`ARG` définit une variable utilisée **pendant le build** de l'image.

DOCKERFILE

```
FROM alpine:3.20
ARG APP_VERSION
RUN echo "Building version $APP_VERSION"
```

TERMINAL

```
docker build --build-arg APP_VERSION=1.0 -t acme-app:1.0 .
```

L'argument est fourni au moment du `docker build`. On peut aussi donner une valeur par défaut : `ARG APP_VERSION=1.0`.

8.2. ENV

`ENV` définit une variable d'environnement disponible **dans l'image et lors de l'exécution** du conteneur.

DOCKERFILE

```
FROM alpine:3.20
ENV APP_VERSION=1.0
CMD ["sh", "-c", "echo Version=$APP_VERSION"]
```

TERMINAL

```
docker build -t acme-app .
docker run --rm acme-app
# Version=1.0
```

8.3. Différence entre ARG et ENV



Figure 7 — ARG vit uniquement pendant le build ; ENV est conservé dans l'image et dans le conteneur.

	ARG	ENV
Utilisation principale	Build	Runtime
Fourni avec	`--build-arg`	`-e` ou `ENV`
Disponible pendant le build	Oui	Oui

	ARG	ENV
Disponible dans le conteneur	Pas automatiquement	Oui
Exemple	version de build	configuration de l'application

Attention — Pas de secrets dans ARG

Ne pas utiliser `ARG` comme mécanisme de stockage de secrets. Les valeurs utilisées pendant un build peuvent être exposées dans l'historique (`docker history`) ou les métadonnées de l'image.

En clair — ARG vs ENV

`ARG` est un échafaudage : utile pendant la construction, démonté ensuite. `ENV` est une inscription gravée dans le bâtiment : elle reste visible une fois celui-ci habité (le conteneur en marche).

À retenir

✓ `ARG` = pendant le build (`--build-arg`) ; `ENV` = dans l'image et le conteneur (`ENV` ou `-e`).
✓ `-e` modifie une variable à l'exécution ; `--build-arg` à la construction.

9. Exemple ARG → ENV

Un cas très fréquent consiste à utiliser un argument de build pour définir une valeur dans l'environnement final.

DOCKERFILE

```
FROM mariadb:latest
ARG ACME_MARIADB_DATABASE
ARG ACME_MARIADB_PASSWORD
ENV MARIADB_DATABASE=${ACME_MARIADB_DATABASE}
ENV MARIADB_ROOT_PASSWORD=${ACME_MARIADB_PASSWORD}
```

TERMINAL

```
docker build \
  --build-arg ACME_MARIADB_DATABASE=acme \
  --build-arg ACME_MARIADB_PASSWORD=acme \
  -t acme:5000/acme-mariadb:latest .
```

En clair — Nom d'image : registry:port/nom:tag

`acme:5000/acme-mariadb:latest` se lit : le registry `acme` sur le port `5000`, l'image `acme-mariadb`, le tag (version) `latest`. Ce nom sert à la fois à identifier l'image et à dire où l'envoyer (`docker push`).

Attention — Exemple pédagogique, pas une bonne pratique

Ici le mot de passe est inscrit dans l'image via `ENV` : quiconque peut lire l'image (`docker inspect`, `docker history`) peut le voir. C'est acceptable pour un exercice, pas en production : on préfère `-e` ou `--env-file` au lancement du conteneur, un secret Docker / Compose, ou les secrets de build BuildKit (`docker build --secret`).

10. Les réseaux Docker

Les applications multi-conteneurs ont généralement besoin de communiquer entre elles. On peut créer un réseau :

```
TERMINAL
docker network create net
docker network ls
```

Les principaux drivers

Driver	Rôle
`bridge`	Réseau privé virtuel sur la machine (le défaut). Les conteneurs y communiquent, isolés du reste.
`host`	Le conteneur partage directement le réseau de l'hôte : aucun isolement, aucun `-p` nécessaire (Linux).
`none`	Aucun réseau : conteneur totalement isolé.
`overlay`	Réseau couvrant plusieurs machines (Docker Swarm).

Réseau par défaut ou réseau personnalisé ?

	Réseau bridge par défaut	Réseau personnalisé
Création	Automatique	`docker network create`
Joindre un conteneur par son **nom**	Non (adresse IP seulement)	Oui (DNS intégré)
Isolement	Tous les conteneurs sans `--network` y sont réunis	Seuls les membres du réseau communiquent

Pourquoi mettre les conteneurs sur le même réseau ?

Prenons une application : WordPress doit contacter MariaDB. Si les deux conteneurs sont connectés au même réseau personnalisé, ils communiquent via le réseau interne et utilisent le **nom du conteneur** comme nom logique.

```
TERMINAL
docker run -d --name mariadb   --network net mariadb
docker run -d --name wordpress --network net wordpress
```

L'application WordPress peut alors utiliser `mariadb` comme hostname de la base de données.

> **En clair — Réseau et nom de conteneur**
> Un réseau Docker personnalisé est comme un réseau d'entreprise avec annuaire : sur le même réseau, on joint un collègue par son nom (`mariadb`) sans connaître son numéro (adresse IP). Sur des réseaux différents, les conteneurs sont comme deux entreprises sans annuaire commun. Sur le réseau par défaut, la résolution par nom n'existe pas : c'est pourquoi on crée son propre réseau.

Pourquoi ne pas utiliser localhost ?

Dans un conteneur, `localhost` désigne le **conteneur lui-même**, pas un autre conteneur.

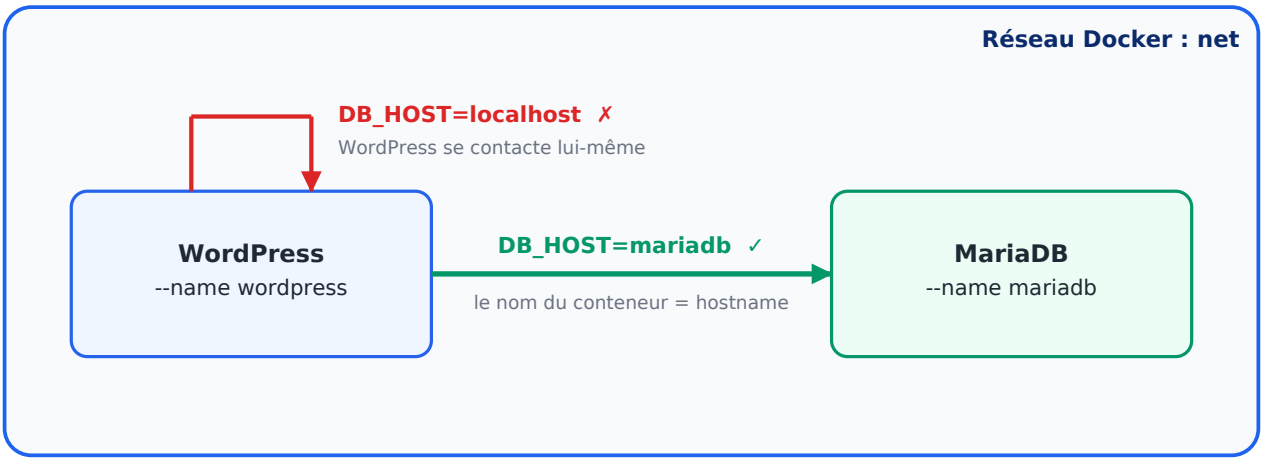


Figure 8 — Même réseau : on joint MariaDB par son nom ; localhost ramène à WordPress lui-même.

Exemple complet (illustratif)

Les images officielles exigent quelques variables pour démarrer (mots de passe d'exemple à ne jamais réutiliser en production).

TERMINAL

```
docker network create net

docker run -d --name mariadb --network net \
  -e MARIADB_ROOT_PASSWORD=exemple \
  -e MARIADB_DATABASE=wordpress \
  mariadb

docker run -d --name wordpress --network net -p 8080:80 \
  -e WORDPRESS_DB_HOST=mariadb \
  -e WORDPRESS_DB_USER=root \
  -e WORDPRESS_DB_PASSWORD=exemple \
  wordpress
```

À retenir

- ✓ Les conteneurs d'un même réseau **personnalisé** se joignent par leur nom.
- ✓ `localhost` = le conteneur lui-même, jamais un autre conteneur.
- ✓ Créer le réseau : `docker network create` ; l'utiliser : `--network`.

11. Docker Compose : le multi-conteneurs simplifié

L'exemple WordPress + MariaDB demande déjà plusieurs commandes longues. **Docker Compose** décrit toute l'application dans un seul fichier `compose.yaml` et la lance d'une seule commande.

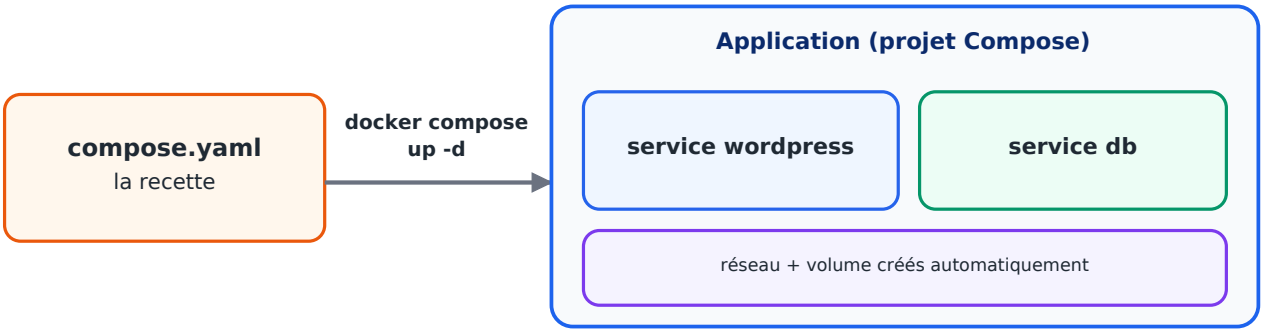


Figure 9 — Compose lit la recette et crée les services, le réseau et le volume.

COMPOSE.YAML

```yaml
services:
  db:
    image: mariadb:11
    environment:
      MARIADB_ROOT_PASSWORD: exemple
      MARIADB_DATABASE: wordpress
    volumes:
      - db-data:/var/lib/mysql

  wordpress:
    image: wordpress
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: root
      WORDPRESS_DB_PASSWORD: exemple
    depends_on:
      - db

volumes:
  db-data:
```

Commande	Rôle
`docker compose up -d`	Créer et démarrer tous les services en arrière-plan
`docker compose ps`	Lister les services de l'application
`docker compose logs -f`	Suivre les logs de tous les services
`docker compose exec db sh`	Ouvrir un shell dans un service
`docker compose stop / start`	Arrêter / redémarrer sans supprimer
`docker compose down`	Supprimer conteneurs et réseau (les volumes restent)
`docker compose down -v`	Supprimer aussi les volumes (**données perdues**)

En clair — Compose = la recette de toute l'application

Compose crée pour vous un réseau personnalisé : chaque **nom de service** (`db`, `wordpress`) devient un hostname. C'est pourquoi `WORDPRESS_DB_HOST: db` fonctionne sans rien configurer.

Attention

`depends_on` contrôle l'**ordre de démarrage**, pas le fait que la base soit prête à répondre. La commande moderne est `docker compose` (sans tiret) ; l'ancien `docker-compose` est l'ancienne version.

À retenir

✓ Un fichier `compose.yaml` décrit services, réseaux et volumes.

✓ `docker compose up -d` lance tout ; `docker compose down` supprime tout (sauf les volumes).

12. Autres commandes essentielles à connaître

Conteneurs

Commande	Rôle
`docker ps / docker ps -a`	Lister les conteneurs en cours / tous
`docker run`	Créer et démarrer un conteneur à partir d'une image
`docker start`	Démarrer un conteneur existant arrêté
`docker stop`	Arrêter un conteneur (il n'est pas supprimé)
`docker restart`	Arrêter puis redémarrer
`docker rm`	Supprimer un conteneur (`-f` pour forcer)
`docker inspect`	Afficher tous les détails (JSON) d'un conteneur ou d'une image
`docker logs`	Afficher la sortie du processus (`-f` pour suivre en direct)
`docker exec`	Exécuter une commande dans un conteneur en cours (`-it ... sh` pour y entrer)
`docker cp`	Copier des fichiers entre hôte et conteneur
`docker stats / docker top`	Consommation CPU/mémoire / processus du conteneur

Images

Commande	Rôle
`docker images`	Lister les images locales
`docker pull`	Télécharger une image depuis un registry
`docker build`	Construire une image depuis un Dockerfile
`docker tag`	Donner un autre nom / tag à une image
`docker push`	Envoyer une image vers un registry (après `docker login`)
`docker history`	Afficher les couches d'une image
`docker rmi`	Supprimer une image

Volumes

Commande	Rôle
`docker volume create`	Créer un volume
`docker volume ls`	Lister les volumes
`docker volume inspect`	Détails d'un volume (emplacement, driver…)
`docker volume rm`	Supprimer un volume
`docker volume prune`	Supprimer les volumes inutilisés

Réseaux

Commande	Rôle
`docker network create`	Créer un réseau
`docker network ls`	Lister les réseaux
`docker network inspect`	Détails d'un réseau (sous-réseau, conteneurs connectés…)
`docker network connect`	Connecter un conteneur existant à un réseau
`docker network disconnect`	Déconnecter un conteneur d'un réseau
`docker network rm`	Supprimer un réseau

Système

Commande	Rôle
`docker version / docker info`	Versions du client et du daemon / état général
`docker system df`	Espace disque utilisé par images, conteneurs, volumes
`docker system prune`	Supprimer conteneurs arrêtés, réseaux et images « dangling » inutilisés

> **Attention**
> `docker system prune -a` supprime **toutes** les images inutilisées, et `--volumes` y ajoute les volumes : vérifiez avant de lancer.

Ce qu'il faut absolument retenir

Je veux…	Je fais…
Construire une image	`docker build -t nom .` (avec un Dockerfile)
Lancer en arrière-plan avec un nom	`docker run -d --name web image`
Accéder à un service depuis l'hôte	`-p HOTE:CONTENEUR` (ex. `-p 8001:80`)
Exposer un dossier de l'hôte (SELinux)	`-v $HOME/site:/chemin:Z`
Partager un volume entre conteneurs (SELinux)	`-v $HOME/data:/data:z`
Conserver des données gérées par Docker	`docker volume create` puis `-v nom:/data`
Copier un fichier vers / depuis un conteneur	`docker cp SRC CTN:DEST` / `docker cp CTN:SRC DEST`
Passer une configuration à l'exécution	`-e NOM=valeur` ou `--env-file`
Paramétrer la construction	`ARG` + `--build-arg`
Faire communiquer deux conteneurs	`docker network create` + `--network` (joindre par le nom)
Lancer toute une application	`docker compose up -d`
Voir les logs / entrer dans un conteneur	`docker logs nom` / `docker exec -it nom sh`

Erreurs fréquentes

✗ Inverser `HOTE:CONTENEUR` dans `-p`.

✗ Croire qu'`EXPOSE` publie le port.

✗ Utiliser `localhost` pour joindre un autre conteneur.

✗ Compter sur la résolution par nom sur le réseau bridge par défaut.

✗ Mettre des secrets dans un `ARG` (ou les graver dans l'image avec `ENV`).

✗ Oublier `:Z` / `:z` sur un système avec SELinux.

✗ Publier deux conteneurs sur le même port hôte.

✗ Stocker des données importantes uniquement dans le conteneur (sans volume).

✗ Lancer `docker compose down -v` sans réaliser que cela supprime les données.

Glossaire des termes importants

Terme	Définition simple
Dockerfile	Fichier décrivant comment construire une image.
Image	Modèle en lecture seule, composé de couches, à partir duquel on crée des conteneurs.
Conteneur	Instance en cours d'exécution (ou arrêtée) d'une image.
Daemon (dockerd)	Service permanent qui gère images, conteneurs, réseaux et volumes.
Client (docker)	Commande qui envoie les ordres au daemon.
Registry	Serveur qui stocke et distribue des images (Docker Hub, quay.io…).
Tag	Étiquette de version d'une image (`latest`, `1.0`…).
Port mapping	Redirection d'un port de l'hôte vers un port du conteneur (`-p`).
Détaché (-d)	Conteneur exécuté en arrière-plan.
Volume	Espace de stockage dont la durée de vie est indépendante de celle du conteneur.
Bind mount	Montage d'un chemin existant de l'hôte dans le conteneur.
Named volume	Volume nommé, dont le stockage est géré par Docker.
Anonymous volume	Volume créé sans nom explicite.
tmpfs	Montage en mémoire vive, temporaire.
SELinux	Système de contrôle d'accès supplémentaire basé sur des étiquettes (contextes).
Relabel (:Z, :z)	Réétiquetage SELinux du volume : `:Z` privé (un conteneur), `:z` partagé.
Variable d'environnement	Paire `NOM=valeur` transmise au processus pour le configurer.
ARG	Variable disponible uniquement pendant le build.
ENV	Variable conservée dans l'image et disponible dans le conteneur.
Réseau bridge	Réseau virtuel privé sur la machine ; les conteneurs y communiquent.
Hostname	Nom d'une machine sur le réseau ; ici, le nom du conteneur.
localhost	Désigne toujours la machine / le conteneur courant.
Éphémère	Qui disparaît avec le conteneur : le filesystem interne du conteneur est éphémère.
Docker Compose	Outil qui décrit et lance une application multi-conteneurs depuis un fichier `compose.yaml`.
Service (Compose)	Un composant de l'application Compose, lancé comme un ou plusieurs conteneurs.

Questions de compréhension — avec corrigés

Essayez de répondre de vous-même avant de lire la correction.

Question 1 Quelle est la différence entre un port du conteneur et un port de l'hôte ?

Correction

Le port du conteneur est celui sur lequel l'application écoute à l'intérieur du conteneur (nginx : 80), dans son réseau isolé. Le port de l'hôte est celui de la machine, par lequel on accède au service depuis l'extérieur. Le port mapping (`-p`) relie les deux.

Question 2 Que signifie `-p 8001:80` ?

Correction

Le port 8001 de l'hôte est redirigé vers le port 80 du conteneur. Ordre : `HOST_PORT:CONTAINER_PORT`. On accède alors à nginx avec `curl http://localhost:8001`.

Question 3 Pourquoi `EXPOSE 80` ne suffit-il pas pour accéder à Nginx depuis l'hôte ?

Correction

`EXPOSE` ne fait que documenter le port utilisé par l'application ; il ne le publie pas. Il faut utiliser `-p` (par exemple `-p 8080:80`) au lancement du conteneur. (L'option `-P` publie tous les ports exposés, mais sur des ports hôtes aléatoires.)

Question 4 Quelle est la différence entre un bind mount et un named volume ?

Correction

Un bind mount monte un chemin existant de l'hôte (`-v /home/student/data:/data`) : l'administrateur le gère directement. Un named volume est géré par Docker, identifié par son nom (`-v acme-data:/data`) : Docker s'occupe du stockage physique.

Question 5 Pourquoi utiliser un volume pour une base de données ?

Correction

Parce que le filesystem du conteneur est éphémère : sans volume, les données sont perdues quand le conteneur est supprimé. Un volume les conserve indépendamment du cycle de vie du conteneur, permet de remplacer ou mettre à jour le conteneur sans perte et facilite les sauvegardes.

Question 6 Quelle est la différence entre `docker cp` et `-v` ?

Correction

`docker cp` fait une copie ponctuelle : les deux copies deviennent indépendantes. `-v` crée un montage permanent défini à la création du conteneur : les mêmes données sont visibles des deux côtés, en temps réel, et persistent après suppression du conteneur.

Question 7 Dans quel sens fonctionne `docker cp` ?

Correction

Dans les deux sens. Host → Container : `docker cp SOURCE CONTAINER:DESTINATION`. Container → Host : `docker cp CONTAINER:SOURCE DESTINATION`. Le côté qui contient `NOM_DU_CONTENEUR:` est celui du conteneur.

Question 8 Quelle est la différence entre ARG et ENV ?

Correction

`ARG` est une variable utilisée pendant le build (fournie avec `--build-arg`), non disponible automatiquement dans le conteneur. `ENV` est une variable d'environnement disponible pendant le build, dans l'image et dans le conteneur (fournie par `ENV` ou `-e`).

Question 9 Quand utilise-t-on `--build-arg` ?

Correction

Au moment de `docker build`, pour paramétrer la construction de l'image (version à installer, nom…), à condition que le Dockerfile déclare l'`ARG` correspondant.

Question 10 Quand utilise-t-on `-e` ?

Correction

Au moment de `docker run`, pour configurer l'application à l'exécution (message, adresse de base de données…) sans reconstruire l'image.

Question 11 Pourquoi ne faut-il pas utiliser ARG pour stocker des secrets ?

Correction

Parce que les valeurs utilisées pendant un build peuvent être exposées dans l'historique (`docker history`) ou les métadonnées de l'image : toute personne pouvant lire l'image pourrait les récupérer.

Question 12 Pourquoi deux conteneurs ne peuvent-ils pas publier simultanément le même port de l'hôte ?

Correction

Parce qu'un port de l'hôte ne peut être écouté que par un seul processus à la fois. Le second conteneur échouerait (port déjà utilisé). Solution : publier sur des ports hôtes différents (`-p 8001:80` et `-p 8002:80`). À l'intérieur, les deux peuvent écouter sur le port 80 car leurs réseaux sont isolés.

Question 13 Pourquoi WordPress et MariaDB doivent-ils être sur le même réseau ?

Correction

Pour qu'ils puissent communiquer via le réseau interne et que WordPress joigne la base en utilisant le nom du conteneur (`mariadb`) comme hostname.

Question 14 Pourquoi `localhost` ne permet-il pas à WordPress de joindre MariaDB ?

Correction

Dans un conteneur, `localhost` désigne le conteneur lui-même (chaque conteneur a son propre namespace réseau). `DB_HOST=localhost` ferait donc contacter WordPress à lui-même ; il faut `DB_HOST=mariadb`.

Question 15 Quel est le rôle de `docker network inspect` ?

Correction

Il affiche les détails d'un réseau : nom, driver, sous-réseau, passerelle, conteneurs connectés… Utile pour diagnostiquer un problème de communication.

Question 16 Quel est le rôle de `docker volume inspect` ?

Correction

Il affiche les détails d'un volume : nom, driver, date de création et surtout son emplacement sur l'hôte (`Mountpoint`).

Question 17 Comment vérifier les logs d'un conteneur ?

Correction

Avec `docker logs NOM` (option `-f` pour suivre en direct, `--tail N` pour les N dernières lignes).

Question 18 Comment entrer dans un conteneur en cours d'exécution ?

Correction

Avec `docker exec -it NOM sh` (ou `bash` si disponible) : `-i` garde l'entrée ouverte, `-t` fournit un terminal.

Question 19 Quelle est la différence entre `docker stop` et `docker rm` ?

Correction

`docker stop` arrête le processus du conteneur mais le conteneur existe encore (visible avec `docker ps -a`, redémarrable avec `docker start`). `docker rm` supprime le conteneur (qui doit être arrêté, ou `-f`). Les données de son filesystem interne sont perdues ; les volumes nommés, eux, restent.

Question 20 Quelle est la différence entre une image et un conteneur ?

Correction

Une image est un modèle en lecture seule (créé par `docker build` ou `docker pull`). Un conteneur est une instance créée à partir de cette image (`docker run`), avec sa propre couche modifiable. Une image peut servir à lancer plusieurs conteneurs.

Question 21 Quel est le rôle du daemon Docker et pourquoi son accès est-il sensible ?

Correction

Le daemon `dockerd` exécute réellement les ordres du client (images, conteneurs, réseaux, volumes). Il tourne en root par défaut : celui qui peut lui parler (groupe `docker`) peut, par exemple, monter le disque de l'hôte dans un conteneur, ce qui équivaut à un accès root.

Question 22 Quelle est la différence entre le réseau bridge par défaut et un réseau personnalisé ?

Correction

Sur le réseau par défaut, les conteneurs ne se joignent que par adresse IP. Sur un réseau créé avec `docker network create`, un DNS intégré permet de joindre un conteneur par son nom, et seuls les membres du réseau communiquent entre eux.

Question 23 À quoi sert Docker Compose ?

Correction

À décrire une application multi-conteneurs (services, réseaux, volumes) dans un fichier `compose.yaml` et à la lancer avec `docker compose up -d`, au lieu d'enchaîner de nombreuses commandes `docker run`. Compose crée un réseau où chaque nom de service sert de hostname.

Question 24 Quelle est la différence entre `docker compose down` et `docker compose down -v` ?

Correction

`down` supprime les conteneurs et le réseau du projet mais conserve les volumes (donc les données).
`down -v` supprime aussi les volumes : les données sont perdues.